

Table of Contents

Week 12 — Object Relationships: The HAS-A Relationship

Composition: Building Objects Out of Other Objects

Lesson Overview

Hour Plan

Recap of Week 11 (4 min)

Part 1 — What Is HAS-A? Objects Made of Other Objects (11 min)

Open with a story (3 min)

Definition

Three Real-World HAS-A Scenarios

The One-Sentence Test

What HAS-A Is NOT

Part 2 — Building Composition: An Object as a Field (11 min)

Open with a story (2 min)

The Core Pattern

Walking Through the Car Example

Part 3 — Delegation: Letting the Part Do the Work (10 min)

Open with a story (2 min)

Delegation in Plain English

Why Delegation Matters

Part 4 — HAS-A vs IS-A: Choosing the Right Relationship (9 min)

Open with a comparison (2 min)

Side-by-Side Comparison

The Decision Rule

Part 5 — Java as Illustration (8 min)

Reading Composition as a Concept

Using the Composed Object

Reading as Plain English

Test Checkpoint

Common Misconceptions

Extension Challenges

● Basic — IS-A or HAS-A?

● Intermediate — Trace the Delegation

Advanced — Design a Composed System

Lesson Summary

Homework (Prep for Static Members, Week 13)

Week 12 — Object Relationships: The HAS-A Relationship

Composition: Building Objects Out of Other Objects

Date: Wednesday, 19 August 2026 | **Time:** 6:30 PM – 7:30 PM | **Course:** Object-Oriented Programming Concepts

Lesson Overview

Item	Detail
Topic	Object Relationships — the HAS-A relationship (composition), and how it differs from the IS-A relationship (inheritance)
Date & Time	Wednesday, 19 August 2026 · 6:30 PM – 7:30 PM
Duration	1 hour
Format	Online (Google Meet) — Concept explanation + real-world discussion + light illustration
Prerequisites	Week 6 (Inheritance — IS-A), Week 11 (Interfaces — “ElectricCar needs a Battery object as a real part of itself”)
Focus	OOP concept understanding — Java used only as illustration
Outcome	Students can define HAS-A (composition) as one object genuinely containing another as a real part of itself, distinguish HAS-A from IS-A, explain delegation (an object using its parts to do work), and apply a clear decision rule for choosing composition over inheritance

Hour Plan

00:00 – 00:04	Recap of Week 11 + Homework check
00:04 – 00:15	Part 1 – What Is HAS-A? Objects Made of Other Objects

00:36 – 00:45	Part 4 – HAS-A vs IS-A: Choosing the Right Relationship
00:45 – 00:53	Part 5 – Java as Illustration (light)
00:53 – 01:00	Checkpoint + Homework

 **Google Meet tip:** Keep *Week12_Summary_Examples.md* open in a second tab — it has a completely different set of real-world stories (a computer's internal parts and a restaurant's kitchen) ready for every part of this lesson.

Recap of Week 11 (4 min)

Ask students verbally:

1. "What is an interface, in one sentence?"
→ *A pure contract that requires certain actions, with zero shared behavior of its own.*
2. "From homework — your `ElectricCar` implemented `Chargeable`, but to actually track its charge level, it needed something more. What did we say it needed?"
→ *A separate `Battery` object — something the `ElectricCar` genuinely CONTAINS as one of its real parts.*
3. "Is `ElectricCar` a `Battery`? Would it make sense to say 'ElectricCar IS-A Battery,' the way we'd say 'Circle IS-A Shape'?"
→ *No — that sentence doesn't make sense at all. An `ElectricCar` isn't a type of `Battery`; it simply HAS a `Battery` as one of its parts.*
4. Transition question: "So if 'IS-A' is the wrong word for this relationship, what IS the right word for 'an object genuinely containing another object as one of its real parts'? That relationship has its own name, and it's today's entire lesson: HAS-A."

Part 1 — What Is HAS-A? Objects Made of Other Objects (11 min)

Open with a story (3 min)

"Think about a car. A car HAS an engine. A car HAS four wheels. A car HAS a battery. None of these are 'family' relationships — you would never say 'a car IS-A engine' or 'a car IS-A wheel.' The car isn't a TYPE of engine — it CONTAINS an engine as one of its real, physical parts.

Take the engine out, and the car isn't a car anymore — it's broken. That's the difference between HAS-A and IS-A: IS-A is about what category something belongs to. HAS-A is about what something is actually built from."

Definition

HAS-A (composition) is the OOP relationship where one object genuinely contains another object as a real part of itself. Unlike IS-A (inheritance), which describes family/category membership, HAS-A describes what an object is physically or logically made of.

Three Real-World HAS-A Scenarios

Work through each one. Ask: "Does the whole object genuinely CONTAIN the part, or is this actually a family relationship in disguise?"

SCENARIO 1 – CARS (continuing from Week 11)

Whole: Car
Has-A: Engine, Battery, Wheel[]

Test: "Car IS-A Engine"? NO – doesn't make sense
"Car HAS-A Engine"? YES – the car genuinely contains one

SCENARIO 2 – HOUSES

Whole: House
Has-A: Room[], Kitchen, Roof

Test: "House IS-A Room"? NO – doesn't make sense
"House HAS-A Room"? YES – the house genuinely contains rooms

SCENARIO 3 – UNIVERSITY (back to our course's own domain)

Whole: University
Has-A: Department[], Student[], Library

Test: "University IS-A Student"? NO – doesn't make sense
"University HAS-A Student"? YES – students are enrolled parts of the university, not a type of university

The One-Sentence Test

THE IS-A / HAS-A TEST:

Say the sentence out loud both ways:

"[Whole] IS-A [Part]" → sounds wrong, absurd
"[Whole] HAS-A [Part]" → sounds natural, correct

If HAS-A sounds right and IS-A sounds absurd, this is composition – the object CONTAINS the other object as a real part, not a family relationship at all.

What HAS-A Is NOT

NOT HAS-A:

A category/family relationship, even if worded loosely
→ "Student HAS-A Person" sounds tempting, but really a Student IS-A Person (Week 6) – this is inheritance, not composition. Watch for this exact trap.

Simply using another object briefly, without truly owning it
→ borrowing a value for one calculation isn't the same as genuinely containing an object as a permanent part

HAS-A:

The whole object genuinely, persistently CONTAINS the part as one of its own real components – not a passing use, and not a family/category relationship.

Part 2 — Building Composition: An Object as a Field (11 min)

Open with a story (2 min)

"When you assemble a car on a factory line, you don't build the engine's internal parts from scratch every time the frame moves down the line — you install an ALREADY-BUILT engine into the car. The car's blueprint simply says: 'this car HAS an engine slot, and here is what engine goes in it.'"

That's exactly how composition works in code: the containing object holds a reference to an already-built object, as one of its own fields."

The Core Pattern

A CLASS "HAS-A" ANOTHER CLASS WHEN:

1. The containing class declares a field whose TYPE is another class (not a primitive like int or String)
2. The contained object is usually created when the containing object is created (often inside the constructor)
3. The containing object can use the part's own actions by calling methods on that field

Walking Through the Car Example

Car (the whole)
has a field: Engine engine

When a Car is built:
the Car's constructor creates (or receives) an Engine
and stores it in the engine field

From now on:
"this Car's engine" always refers to that SAME Engine
object – it is a real, permanent part of this specific
Car, not a temporary value

CONTRAST WITH INHERITANCE (Week 6):

INHERITANCE (IS-A)

Circle extends Shape

Circle automatically GETS everything

Shape has, built into its own type

COMPOSITION (HAS-A)

Car has a field: Engine engine

Car must explicitly CREATE

or receive an Engine object

and store it as a separate
part

Part 3 — Delegation: Letting the Part Do the Work (10 min)

Open with a story (2 min)

"When you turn a car's key, do YOU personally ignite the fuel? No — the car's ignition system asks the ENGINE to start, and the engine does the actual work of igniting fuel. The car doesn't need to know HOW an engine ignites fuel internally — it just needs to know 'call start() on my engine, and trust it to handle the details.'"

This is called delegation: the whole object doesn't do the work itself — it delegates the work to the part it contains."

Delegation in Plain English

Delegation is when a containing object performs an action by calling a method on one of the objects it HAS, instead of doing all the work itself.

EXAMPLE – Car.start()

Car.start() does NOT contain the actual ignition logic.

Instead, Car.start() calls:

engine.start()

The Engine object – the PART – does the real work of actually starting. The Car simply delegates the job to the part that knows how to do it.

Why Delegation Matters

THE PAYOFF:

Because the Car delegates to its Engine, you could later swap in a completely different Engine (say, a more powerful one) WITHOUT changing a single line of Car's own code – as long as the new Engine also has a working `start()` method.

The Car doesn't care HOW the engine starts. It only trusts that calling `start()` on its engine field will work – the exact same trust polymorphism (Week 9) is built on, just applied to a "part" instead of a "child."

Part 4 — HAS-A vs IS-A: Choosing the Right Relationship (9 min)

Open with a comparison (2 min)

"This is the single most important design decision in basic OOP: for any two classes, should they be connected by IS-A (inheritance) or HAS-A (composition)? Choosing wrong leads to confusing, broken designs."

Side-by-Side Comparison

IS-A (Inheritance, Week 6)	HAS-A (Composition, today)
Student IS-A Person → Student is a MORE SPECIFIC kind of Person – same category, just more detailed	Car HAS-A Engine → Car CONTAINS an Engine as one of its parts – totally different kind of thing
Sentence test: "Student IS-A Person" sounds correct and natural	Sentence test: "Car HAS-A Engine" sounds correct; "Car IS-A Engine" sounds absurd
Student automatically inherits Person's behavior as part of its OWN type	Car must explicitly create or receive an Engine and store it as a separate field

The Decision Rule

ASK YOURSELF, IN ORDER:

1. Say both sentences out loud:
"[A] IS-A [B]" and "[A] HAS-A [B]"
2. If "IS-A" sounds natural and B is a more general VERSION of A (same basic kind of thing, just broader) → use INHERITANCE
3. If "HAS-A" sounds natural and B is a completely DIFFERENT kind of thing that A merely contains or owns → use COMPOSITION

COMMON TRAP: "Car IS-A Engine" is WRONG (sounds absurd) – even though a car needs an engine, they are not the same KIND of thing. This should always be HAS-A.

Part 5 — Java as Illustration (8 min)

Reminder: Java is just how we write these concepts down. Focus on the concept, not the syntax.

Reading Composition as a Concept

```
// THE PART – a complete, independent class
public class Engine {
    private int horsepower;

    public Engine(int horsepower) {
        this.horsepower = horsepower;
    }

    public void start() {
        System.out.println("Engine (" + horsepower + " hp) roars to life");
    }
}

public class Battery {
    private int capacityKwh;
```

```

        public Battery(int capacityKwh) {          this.capacityKwh = capacityKwh;      }
        public void charge() {
            System.out.println("Battery (" + capacityKwh + " kWh) is charging");    }    }
        // THE WHOLE - HAS-A Engine, HAS-A Battery
public class Car {    private String model;
private Engine engine;    // HAS-A Engine - a field of another class
private Battery battery;    // HAS-A Battery - a second field
public Car(String model, int horsepower, int capacityKwh) {          this.model = model;
    this.engine = new Engine(horsepower);    // built when the Car is built
    this.battery = new Battery(capacityKwh);    }          public void start() {
        System.out.println(model + " is starting ...");
        engine.start();    // DELEGATION - Car asks its Engine to do the work    }
        public void chargeBattery() {
            battery.charge();    // DELEGATION - Car asks its Battery to do the work    }
    }
}

```

Using the Composed Object

```

Car myCar = new Car("Toyota Camry", 200, 0);
myCar.start();

```

OUTPUT:

```

Toyota Camry is starting ...
Engine (200 hp) roars to life

```

Car did not personally know HOW to "roar to life" - it delegated that entire job to the Engine object it HAS.

Reading as Plain English

CONCEPT TRANSLATION TABLE

Java code	OOP concept meaning
private Engine engine;	A field whose type is another class - this IS the HAS-A relationship in code
this.engine = new Engine(...); (inside the constructor)	The part is built and stored as a permanent piece of the whole, at creation time
engine.start(); (inside Car.start())	DELEGATION - the whole calls a method on the part instead of doing the work itself

Test Checkpoint

- ☐ Can define HAS-A (composition) as one object genuinely containing another as a real part of itself
 - ☐ Can apply the IS-A/HAS-A sentence test to correctly classify a relationship between two classes
 - ☐ Can explain delegation as “the whole calls a method on the part instead of doing the work itself”
 - ☐ Can explain why “Student HAS-A Person” is a common trap, and why it should actually be IS-A
 - ☐ Can apply the full decision rule to choose between inheritance and composition for a new pair of classes
-

Common Misconceptions

Misconception	Clarification
“Any two classes that work together must have a HAS-A relationship”	No — always run the sentence test first. If IS-A sounds natural and B is a broader version of the same kind of thing as A, it’s inheritance, not composition.
“Student HAS-A Person, because a Student ‘has’ a name and age like a Person does”	This is the most common trap. A Student is not a container that owns a separate Person object — a Student genuinely IS a (more specific kind of) Person. This is IS-A (Week 6), not HAS-A.
“Composition means copying all the part’s code into the whole class”	No — the whole does NOT copy the part’s code. It stores a reference to a real, separate object and calls that object’s own methods (delegation) — the part’s code stays entirely inside the part’s own class.
“Delegation means the whole object still does the actual work, just organized differently”	No — delegation means the PART does the real work. The whole object’s method is often just one line: calling the part’s method and trusting it to handle the details.
“You should always prefer inheritance because it’s more powerful”	Composition is often the SAFER, more flexible choice, especially when the sentence test fails for IS-A. Using inheritance where composition belongs (forcing an “IS-A” that doesn’t really make sense) leads to confusing, broken designs.

Extension Challenges

● Basic — IS-A or HAS-A?

For each pair below, run the sentence test and decide: is this IS-A (inheritance) or HAS-A (composition)?

1. `Laptop` and `Keyboard`
2. `Manager` and `Employee`
3. `Library` and `Book`
4. `SavingsAccount` and `BankAccount`
5. `Order` and `Customer`

● Intermediate — Trace the Delegation

Given: `class Library` HAS-A `Catalog catalog`, and `Catalog` has a method `findBook(title)`.

```
public class Library {  
    private Catalog catalog;  
  
    public Library() {  
        this.catalog = new Catalog();  
    }  
  
    public void searchFor(String title) {  
        catalog.findBook(title); // delegation  
    }  
}
```

1. Which object actually performs the real search logic — `Library` or `Catalog`?
2. If a completely different, smarter `Catalog` implementation is swapped in later, does `Library.searchFor()` need to change? Why or why not?
3. Rewrite `searchFor()` in plain English if `Library` did NOT delegate, and explain why that version would be worse design.

● Advanced — Design a Composed System

Design (in plain English, no code required) a `SmartHome` class that HAS-A `Thermostat`, HAS-A `SecuritySystem`, and HAS-A `LightingSystem`.

- For each part, write ONE method on `SmartHome` that delegates to that part (e.g., `SmartHome.setTemperature()` delegates to `Thermostat`).

- Explain why none of these three parts should instead be modeled using inheritance (i.e., why "SmartHome IS-A Thermostat" would be the wrong design).
- Explain what would need to change in `SmartHome` if the `Thermostat` were swapped for a different brand's thermostat — assuming the new thermostat still has a working `setTemperature()` method.

Lesson Summary

TODAY'S KEY TAKEAWAYS

1. HAS-A (composition) means one object genuinely CONTAINS another object as a real part of itself – a completely different kind of relationship from IS-A (inheritance).
2. THE SENTENCE TEST: say "[A] IS-A [B]" and "[A] HAS-A [B]" out loud. Whichever sounds natural tells you the correct relationship.
3. In code, HAS-A appears as a FIELD whose type is another class, usually created inside the containing object's constructor.
4. DELEGATION means the whole object calls a method on the part it contains, instead of doing the real work itself – "ask the part, trust the part."
5. THE #1 TRAP: relationships that LOOK like HAS-A ("Student has a name, like a Person has a name") are often actually IS-A in disguise. Always run the sentence test carefully.
6. Composition is often the safer, more flexible design choice – swapping out a contained part doesn't break the whole object's own code, as long as the new part still provides the same working actions.

Homework (Prep for Static Members, Week 13)

Design the following system in plain English — code is optional (bonus only):

`Library` **HAS-A** `Book[]` **collection:**

1. `Book`: attributes: `title`, `author`, `isbn`

2. `Library`: has a field holding a collection of `Book` objects (HAS-A relationship — a Library genuinely contains its books, it is not a type of Book)
 - `addBook(book)` — adds a new Book to the collection
 - `totalBooks()` — reports how many books the library currently HAS

Run the sentence test out loud and confirm: does “Library IS-A Book” sound wrong, and does “Library HAS-A Book” sound correct? Write one sentence explaining why.

Concept to carry forward: Now imagine TWO different `Library` objects both need to know the TOTAL number of books ever added, across BOTH libraries combined — not just their own individual collection. Each `Library` object has its OWN `totalBooks()` for its own books, but this new number needs to be SHARED and counted across every Library object that exists, everywhere in the program. A value that belongs to the CLASS itself, not to any one individual object, is the seed of our next topic: **Static Members** (Week 13).

Week 12 of 16 | Object-Oriented Programming Concepts | @KhmerSide | syuthd.com